

API EcoDrop

Schéma de la Base de Données

L'API repose sur 5 tables relationnelles :

- wastetype : Catalogue des déchets (id , nom , pointsPerKilo).
- collectionPoint : Points de collecte (id , adresse , capaciteMax).
- accepts : Table d'association liant collectionPoint et wastetype (#pointID , #wasteTypeID).
- deposit : Historique des dépôts liant users , collectionPoint et wastetype (id , #userId , #pointId , #wastetypeId , poids).
 - users : Gère les utilisateurs (id , login , password , role).

Requêtes complexes

Calcul du Leaderboard (Les 10 meilleurs recycleurs) Cette requête calcule le score des utilisateurs pour un point donné.

```
select us.id, us.login, sum(dep.poids*w.pointsPerKilo) as score
from deposit as dep
join users as us on dep.userId=us.id
join wastetype as w on w.id=dep.wastetypeId
where dep.pointId = ?
group by us.id
order by sum(dep.poids*w.pointsPerKilo) desc
limit 10
```

Récupération des types de déchets autorisés dans un point de collecte donné

```
select w.id, w.nom , w.pointsperkilo
from accepts
join wastetype as w on w.id = wastetypeid
join collectionpoint as c on c.id = pointid
where c.id= ?
```

Récupération des informations dur un dépôt Récupération des données des wastetypes liés, et du point de collecte correspondant

```
select dep.id, dep.userId, point.id, point.adresse, point.capaciteMax,
       w.id, w.nom, w.pointsPerKilo ,
       dep.poids , dep.dateDepot, dep.collecte
from deposit as dep
```

```
join collectionpoint as point on dep.pointId = point.id
join wastetype as w on w.id = dep.wastetypeId
```

API Waste-types

URI	Opérations	MIME	Requête	Header	Réponse
/waste-types	GET	<- application/json <- application/xml			liste de tous les types de déchets disponibles (w1 ou w1.b)
/waste-types/{id}	GET	<- application/json <- application/xml			Détail d'un type spécifique (w1 ou w1.b) ou 404
/waste-types	POST	<- application/json	Waste-Type (w1)	Authorization: Basic token (ADMIN)	ajout d'un nouveau type (w1)
/waste-types/{id}	PUT	<- application/json	Waste-Type (w1)	Authorization: Basic token (ADMIN)	mise à jour complète d'un type (w1)
/waste-types/{id}	DELETE			Authorization: Basic token (ADMIN)	supprime un type ou 409

Corps des requêtes

w1

Un waste-type comporte un id, un nom, et ses pointsPerKilo. Sa représentation est la suivante :

```
{
  "id": 2,
  "nom": "carton",
  "pointsPerKilo": 3
}
```

w1.b

```
<item>
  <id>2</id>
  <nom>carton</nom>
  <pointsPerKilo>3</pointsPerKilo>
</item>
```

Exemples

Afficher l'ensemble des types de déchets de la base de donnée

GET /decheterie/waste-types

Requête vers le serveur

```
GET /decheterie/waste-types
```

Requête du serveur

```
<ArrayList xmlns="">
  <item>
    <id>2</id>
    <nom>carton</nom>
    <pointsPerKilo>3</pointsPerKilo>
  </item>
  <item>
    <id>3</id>
    <nom>verre</nom>
    <pointsPerKilo>4</pointsPerKilo>
  </item>
</ArrayList>
```

ou

```
[
  {
    "id": 2,
    "nom": "carton",
    "pointsPerKilo": 3
  },
  {
    "id": 3,
    "nom": "verre",
    "pointsPerKilo": 4
  }
]
```

Codes de status HTTP

Status	Description
200 OK	la requête s'est effectuée correctement

Afficher les informations d'un type précis

GET /decheterie/waste-types/{id}

Requête vers le serveur

```
GET /decheterie/waste-types/1
```

Requête du serveur

```
<Dechets xmlns="">
  <id>1</id>
  <nom>dechets organiques</nom>
  <pointsPerKilo>5</pointsPerKilo>
</Dechets>
```

ou

```
{
  "id": 7,
  "nom": "papier",
  "pointsPerKilo": 5
}
```

Codes de status HTTP

Status	Description
200 OK	la requête s'est effectuée correctement
404 NOT FOUND	le type n'existe pas

Ajouter un nouveau type

POST /decheterie/waste-types/

requête vers le serveur (Header: Authorization: Basic token)

```
POST /decheterie/waste-types/
{
  "id": 6,
```

```
    "nom": "plastique",
    "pointsPerKilo": 8
  }
```

Codes de status HTTP

Status	Description
200 OK	la requête s'est effectuée correctement
401 UNAUTHORIZED	Token manquant ou invalide

Modifier entièrement un type

PUT /decheterie/waste-types/1

requête vers le serveur (Header: Authorization: Basic token)

```
PUT /decheterie/waste-types/1
{
  "id": 1,
  "nom": "dechets organiques",
  "pointsPerKilo": 5
}
```

Codes de status HTTP

Status	Description
200 OK	la requête s'est effectuée correctement
401 UNAUTHORIZED	Token manquant ou invalide
404 NOT FOUND	Le type n'existe pas

Supprimer un type

DELETE /decheterie/waste-types/{id}

requête vers le serveur (Header: Authorization: Basic token)

```
DELETE /decheterie/waste-types/5
```

Codes de status HTTP

Status	Description
200 OK	la requête s'est effectuée correctement
401 UNAUTHORIZED	Token manquant ou invalide
409 CONFLICT	Le type est utilisé dans une autre table

API CollectionPoints

URI	Opérations	MIME	Requête	Header	Réponse
/points	GET	<- application/json			liste de tous les points de collecte (p2)
/points/{id}	GET	<- application/json			Détail d'un point (p1) incluant la liste des WasteTypes imbriqués ou 404
/points/{id}	PATCH	<- application/json	CollectionPoints (p3)	Authorization: Basic token (ADMIN)	modification partielle d'un point (p3) ou 404
/points/{id}	PUT	<- application/json	CollectionPoints (p3)	Authorization: Basic token (ADMIN)	modification partielle d'un point (p3) ou 404
/points/{id}/clear	DELETE			Authorization: Basic token (ADMIN)	vider un point de collecte
/points/{id}/status	GET	<- application/json			récupérer le taux de remplissage d'un point de collecte (p4)
/points/overloaded	GET	<- application/json		Authorization: Basic token (ADMIN)	liste de tous les points de collectes

URI	Opérations	MIME	Requête	Header	Réponse
					remplis (dont le taux est supérieur à 100%) (p4)

Corps des requêtes

p1

Un point de collecte comporte un id, une adresse, une capacité max, une liste de déchets, une liste de topUsers, un taux de remplissage, et un boolean pour savoir si ce point est rempli ou non. Sa représentation est la suivante :

```
{
  "id": 1,
  "adresse": "Roubaix",
  "capaciteMax": 300,
  "listeDechets": [
    {
      "id": 2,
      "nom": "carton",
      "pointsPerKilo": 3
    },
    {
      "id": 1,
      "nom": "dechets organiques",
      "pointsPerKilo": 5
    }
  ],
  "topUsers": [],
  "remplissage": 0.0,
  "full": false
}
```

p2

Dans les informations générales des points de collecte, la liste des wastetypes acceptées, des meilleurs utilisateurs et les informations sur les taux de remplissage ne sont pas affichés. On se retrouve ainsi avec une structure comme celle-ci:

```
{
  "id": 1,
  "adresse": "Roubaix",
  "capaciteMax": 300,
  "listeDechets": [],
  "topUsers": [],
  "remplissage": 0.0,
  "full": false
}
```

```
}
```

p3

Exemple de structure à donner pour une modification partielle d'un point de collecte

```
{
  "adresse": "Lille",
  "capaciteMax": 100
}
```

p4

Structure d'un taux de remplissage d'un point de collecte:

```
[
  {
    "id": 1,
    "adresse": "Roubaix",
    "capaciteMax": 300,
    "listeDechets": [],
    "topUsers": [],
    "remplissage": 110.00000000000001,
    "full": true
  }
]
```

Exemples

Afficher l'ensemble des points de collecte de la base de donnée

GET /decheterie/points

Requête vers le serveur

```
GET /decheterie/points
```

Requête du serveur

```
[
  {
    "id": 1,
    "adresse": "Roubaix",
    "capaciteMax": 300,
    "listeDechets": [],
    "topUsers": [],
```



```
    "remplissage": 0.0,
    "full": false
  },
  {
    "id": 2,
    "adresse": "Annoellin",
    "capaciteMax": 200,
    "listeDechets": [],
    "topUsers": [],
    "remplissage": 0.0,
    "full": false
  }
]
```

Codes de status HTTP

Status	Description
200 OK	la requête s'est effectuée correctement

Afficher les informations d'un point précis

GET /decheterie/points/{id}

Requête vers le serveur

```
GET /decheterie/points/1
```

Requête du serveur

```
{
  "id": 1,
  "adresse": "Roubaix",
  "capaciteMax": 300,
  "listeDechets": [
    {
      "id": 2,
      "nom": "carton",
      "pointsPerKilo": 3
    },
    {
      "id": 1,
      "nom": "dechets organiques",
      "pointsPerKilo": 5
    }
  ],
  "topUsers": [],
  "remplissage": 0.0,
  "full": false
}
```

Codes de status HTTP

Status	Description
200 OK	la requête s'est effectuée correctement
404 NOT FOUND	le point n'existe pas

Modifier partiellement un point de collecte

PATCH /decheterie/points/{id}

PUT /decheterie/points/{id}

requête vers le serveur (Header: Authorization: Basic token)

```
PATCH /decheterie/points/4
{
  "capaciteMax": 100
}
```

ou

```
PUT /decheterie/points/4
{
  "capaciteMax": 100
  "adresse": "Marquettes-lez-Lille"
}
```

Codes de status HTTP

Status	Description
200 OK	la requête s'est effectuée correctement
401 UNAUTHORIZED	Token manquant ou invalide
404 NOT FOUND	Le point n'existe pas

Vider un point de collecte

DELETE /decheterie/points/{id}/clear

requête vers le serveur (Header: Authorization: Basic token)

```
DELETE /decheterie/points/1/clear
```

Codes de status HTTP

Status	Description
200 OK	la requête s'est effectuée correctement
401 UNAUTHORIZED	Token manquant ou invalide
404 NOT FOUND	Le point n'existe pas

Avoir le taux de remplissage d'un point

```
GET /decheterie/points/{id}/status
```

requête vers le serveur

```
GET /decheterie/points/1/status
```

requête du serveur

```
{
  "id": 1,
  "adresse": "Roubaix",
  "capaciteMax": 300,
  "listeDechets": [],
  "topUsers": [],
  "remplissage": 0.0,
  "full": false
}
```

Codes de status HTTP

Status	Description
200 OK	la requête s'est effectuée correctement
404 NOT FOUND	Le type n'existe pas

Avoir la liste des points remplis

```
GET /decheterie/points/overloaded
```

requête vers le serveur (Header: Authorization: Basic token)

```
GET /decheterie/points/overloaded
```

requête du serveur

```
[
  {
    "id": 1,
    "adresse": "Roubaix",
    "capaciteMax": 300,
    "listeDechets": [],
    "topUsers": [],
    "remplissage": 110.00000000000001,
    "full": true
  }
]
```

Codes de status HTTP

Status	Description
200 OK	la requête s'est effectuée correctement
401 UNAUTHORIZED	Token manquant ou invalide
404 NOT FOUND	Le type n'existe pas

API Deposit

URI	Opérations	MIME	Requête	Header	Réponse
/deposits	GET	<- application/json			liste des dépôts
/deposits/{id}	GET	<- application/json			informations sur un déposit précis ou 404
/deposits	POST	<- application/json	Deposit (d1)	Authorization: Basic token (USER)	ajout d'un dépôt
/deposits/{id}	PUT	<- application/json	Deposit (d2)	Authorization: Basic token (ADMIN)	modification partielle d'un dépôt (d2)
/deposits/{id}	PATCH	<- application/json	Deposit (d2)	Authorization: Basic token (ADMIN)	modification partielle d'un dépôt (d2)

Corps des requêtes

d1

Un deposit comporte un id, un userId, les informations (p2) du point de collecte correspondant, les informations sur le type de déchets recyclé, un poids, une date de dépôt et un booléen de vérification de collecte. Sa représentation est la suivante:

```
{
  "id": 1,
  "userId": 1,
  "point": {
    "id": 1,
    "adresse": "Roubaix",
    "capaciteMax": 300,
    "listeDechets": [],
    "topUsers": [],
    "remplissage": 0.0,
    "full": false
  },
  "wastetype": {
    "id": 1,
    "nom": "dechet vert",
    "pointsPerKilo": 6
  },
  "poids": 130,
  "dateDepot": null,
  "collecte": false
}
```

d2

La modification se fera sur une ou plusieurs parties d'un dépôt. Un exemple de représentation de requête partielle est la suivante

```
{
  "id": 8,
  "userId": 3,
  "wastetypeId": 4,
  "poids": 100
}
```

Exemples

Lister l'ensemble des dépôts

GET /decheterie/deposits

Requête vers le serveur

GET /decheterie/deposits

Requête du serveur

```
[
  {
    "id": 1,
    "userId": 1,
    "point": {
      "id": 1,
      "adresse": "Roubaix",
      "capaciteMax": 300,
      "listeDechets": [],
      "topUsers": [],
      "remplissage": 0.0,
      "full": false
    },
    "wastetype": {
      "id": 1,
      "nom": "dechet vert",
      "pointsPerKilo": 6
    },
    "poids": 130
  },
  {
    "id": 2,
    "userId": 1,
    "point": {
      "id": 1,
      "adresse": "Roubaix",
      "capaciteMax": 300,
      "listeDechets": [],
      "topUsers": [],
      "remplissage": 0.0,
      "full": false
    },
    "wastetype": {
      "id": 3,
      "nom": "verre",
      "pointsPerKilo": 4
    },
    "poids": 70
  }
]
```

Codes de status HTTP

Status	Description
200 OK	la requête s'est effectuée correctement

Afficher les informations d'un dépôt précis

```
GET /decheterie/deposits/{id}
```

Requête vers le serveur (Header: Authorization: Basic token)

GET /decheterie/deposits/1

Réponse du serveur

```
{
  "id": 1,
  "userId": 1,
  "point": {
    "id": 3,
    "adresse": "Annoellin",
    "capaciteMax": 200,
    "listeDechets": [],
    "topUsers": [],
    "remplissage": 0.0,
    "full": false
  },
  "wastetype": {
    "id": 5,
    "nom": "papier",
    "pointsPerKilo": 5
  },
  "poids": 30,
  "dateDepot": null,
  "collecte": false
}
```

Codes de status HTTP

Status	Description
200 OK	la requête s'est effectuée correctement
404 NOT FOUND	Le dépôt n'existe pas

Ajouter un dépôt

POST /decheterie/deposits/

requête vers le serveur (Header: Authorization: Basic token)

```
POST /decheterie/deposits/
{
  "id": 9,
  "userId": 1,
  "pointId": 2,
  "wastetypeId": 1,
  "poids": 150
}
```

Codes de status HTTP

Status	Description
200 OK	la requête s'est effectuée correctement
400 BAD REQUEST	Le poids du dépôt est négatif ou le point est introuvable
401 UNAUTHORIZED	Token manquant ou invalide
403 FORBIDDEN	Le point de collecte est saturé
409 CONFLICTS	Ce dépôt existe déjà

Modifier partiellement un dépôt

PUT /decheterie/deposits/{id} **PATCH** /decheterie/deposits/{id}

requête vers le serveur (Header: Authorization: Basic token)

```
PUT /decheterie/deposits/2
{
  "userId": 1,
  "pointId": 2,
}
```

ou

```
PATCH /decheterie/deposits/2
{
  "userId": 1,
  "pointId": 2,
}
```

Codes de status HTTP

Status	Description
200 OK	la requête s'est effectuée correctement
401 UNAUTHORIZED	Token manquant ou invalide

API Users

URI	Opérations	MIME	Requête	Réponse
/users/leaderboard	GET	<- application/json		liste des points de collectes (p1) avec leurs meilleurs recycleurs (u1)
/users/{id}	PUT	<- application/json	Users (u2)	modification partielle d'un User (p1)
/users/{id}	PATCH	<- application/json	Users (u2)	modification partielle d'un User (p1)

Corps des requêtes

u1

Un User comporte un id, un login, un mot de passe, un rôle (USER ou ADMIN), et un score. Sa représentation est la suivante

```
{
  "id": 1,
  "login": "login",
  "password": "password",
  "role": "USER/ADMIN",
  "score": 100
}
```

u2

La modification se fera sur une ou plusieurs parties d'un User, sur ses attributs login et/ou password et/ou role. Un exemple de représentation de requête partielle est la suivante

```
{
  "login": "login",
}
```

Exemples

Lister des 10 meilleurs recycleurs par points de collecte dans la base de donnée

GET /decheterie/users/leaderboard

Requête vers le serveur

```
GET /decheterie/users/leaderboard
```

Requête du serveur

```
[
  {
    "id": 1,
    "adresse": "Roubaix",
    "capaciteMax": 300,
    "listeDechets": [
      {
        "id": 2,
        "nom": "carton",
        "pointsPerKilo": 3
      },
      {
        "id": 1,
        "nom": "dechets organiques",
        "pointsPerKilo": 5
      }
    ],
    "topUsers": [
      {
        "id": 3,
        "login": "tata",
        "password": "tata",
        "role": "USER",
        "score": 700
      },
      {
        "id": 1,
        "login": "toto",
        "password": "toto",
        "role": "ADMIN",
        "score": 630
      },
      {
        "id": 2,
        "login": "tutu",
        "password": "tutu",
        "role": "ADMIN",
        "score": 180
      }
    ],
    "remplissage": 0.0,
    "full": false
  }
]
```

Codes de status HTTP

Status	Description
200 OK	la requête s'est effectuée correctement

Modifier le login d'un User avec PATCH

PATCH /decheterie/users/{id}

requête vers le serveur (Header: Authorization: Basic token)

```
PATCH /decheterie/users/1
{
  "login": "login",
}
```

Codes de status HTTP

Status	Description
200 OK	la requête s'est effectuée correctement
401 UNAUTHORIZED	Token manquant ou invalide
404 NOT FOUND	L'utilisateur n'existe pas

Modifier le password d'un User avec PUT

PUT /decheterie/users/{id}

requête vers le serveur (Header: Authorization: Basic token)

```
PUT /decheterie/users/1
{
  "password": "new_password",
  "role": "ADMIN"
}
```

Codes de status HTTP

Status	Description
200 OK	la requête s'est effectuée correctement
401 UNAUTHORIZED	Token manquant ou invalide
404 NOT FOUND	L'utilisateur n'existe pas

API Auth

URI	Opérations	Réponse
/auth/token?login={login}&password={password}	GET	authentification à la base (en tant que ADMIN ou USER) et récupération d'un jeton de connexion ou 401 si la connexion échoue

Utilisation

Recevoir un token de connexion

```
GET /decheterie/auth/token?login={login}&password={password}
```

Requête vers le serveur

```
GET /decheterie/auth/token?login=toto&password=toto
```

Requête du serveur

```
dG90bzp0b3Rv
```

Codes de status HTTP

Status	Description
200 OK	la requête s'est effectuée correctement
401 UNAUTHORIZED	Utilisateur inconnu dans la base